

Usos y diferencias entre Sleep, Light Sleep y Deep Sleep

Por Ricardo Irv LB

Primero que nada, estos pueden definirse como **modos de bajo consumo**, ya que dependiendo de que tanto uso de memoria se utilice es equivalente al uso de energía que usa la ESP; por lo que estos modos hacen que aparentemente “no haga algo”, pero cada uno de ellos ahorra mucha más energía.

Sleep.

- Este modo detiene cualquier ejecución de programa un tiempo específico
- No ahorra mucha energía (por no decir nada) ya que los componentes siguen activos
- Se usaría principalmente para hacer breves pausas en la ejecución del programa y conservar datos algo extensos

Light Sleep.

- Este modo apaga algunas partes del microprocesador para reducir el consumo de energía; mantiene prendida la memoria RTC y la RAM para despertar rápidamente
- Con este modo si se puede ahorrar significativamente más energía, sin sacrificar tiempos rápidos de respuesta
- Este modo es muy optimo para sensores que necesitan medias periódicas y frecuentes cada cierto tiempo siempre optimizando la energía utilizada

Deep Sleep.

- Este modo apaga casi todos los componentes del microprocesador, excepto la memoria RTC y ciertas partes de la RAM, consumiendo la menor cantidad de energía posible (solo lo esencial)
- Este modo es perfecto para aplicaciones o sensores que requieren tiempos para despertar más largos y necesitan suministrar la energía de la forma más esencial posible.
- También es muy útil para dispositivos IoT que solo necesitan despertarse esporádicamente para enviar datos

Código "SLEEP + Contador"

```
#Sleep + contador
from machine import Pin
from time import sleep

# Botones
sleep_button = Pin(4, Pin.IN, Pin.PULL_UP) # Botón para entrar en modo "sleep"
wake_button = Pin(5, Pin.IN, Pin.PULL_UP) # Botón para continuar el conteo

# Variable para el progreso del contador
contador = 1 # Empezamos desde 1

# Loop principal
while contador <= 100:
    print("Contador:", contador)
    contador += 1 # Incrementa el contador

    # Verifica si el botón de "sleep" se presiona
    if sleep_button.value() == 0: # Botón presionado (nivel bajo)
        print("Entrando en modo de espera...")
        while True:
            # Espera a que se presione el botón para "despertar"
            if wake_button.value() == 0: # Botón presionado (nivel bajo)
                print("Despertando y continuando...")
                sleep(0.2) # Prevención de rebotes
                break

        sleep(1) # Pausa entre números para facilitar la visualización

print("¡El contador ha terminado!")
```

Explicación de código

Contador simple:

- Usamos la variable "contador" para almacenar el progreso del conteo.
- Cada iteración incrementa el contador y lo imprime.

Botón de "sleep":

- Si se presiona sleep_button, el programa entra en un bucle de espera.
- El bucle se mantiene hasta que se presiona el botón de "despertar" (wake_button).

Botón de "despertar":

- Al presionarlo, se sale del estado de espera y el programa continúa contando desde donde se quedó.

Prevención de rebotes:

- Se añade un pequeño retraso (sleep(0.1)) después de detectar un botón presionado para evitar lecturas duplicadas debidas al rebote del hardware del botón.

Sin almacenamiento externo:

- Todo el progreso se almacena en la variable current_count, que existe solo mientras el programa se está ejecutando.

Código "LIGHT SLEEP + Contador"

```
#LightSleep+Contador
from machine import Pin, lightsleep
from time import sleep

# Botones
sleep_button = Pin(4, Pin.IN, Pin.PULL_UP) # Botón para entrar en modo "sleep"
wake_button = Pin(5, Pin.IN, Pin.PULL_UP) # Botón para "despertar"

# Variable para el progreso del contador
contador = 1 # Empezamos desde 1

# Loop principal
while contador <= 100:
    print("Contador:", contador)
    contador += 1 # Incrementa el contador
    sleep(1) # Pausa entre números para facilitar la visualización

    # Verifica si el botón de "sleep" se presiona
    if sleep_button.value() == 0: # Botón presionado (nivel bajo)
        print("Entrando en modo light sleep...")

        # Espera a que se presione el botón de "despertar"
        while wake_button.value() != 0: # Mientras el botón no esté presionado
            lightsleep(100) # Light sleep por 100 ms (menor consumo)

        print("Despertando y continuando...")
        sleep(0.1) # Prevención de rebotes

print("¡El contador ha terminado!")
```

Modo Light Sleep:

- Usamos lightsleep(100) dentro de un bucle para mantener el ESP32 en modo de bajo consumo hasta que se presione el botón de despertar.

Control con botones:

- Cuando se presiona el sleep_button, el programa entra en un bucle de **light sleep**.
- El bucle se rompe cuando se presiona el wake_button para continuar contando.

Uso de ciclos cortos de light sleep:

- El ESP32 entra en ciclos de light sleep de 100 ms. Esto asegura un bajo consumo mientras sigue revisando si el botón de despertar ha sido presionado.

Prevención de rebotes:

- Usamos una pausa breve (sleep(0.1)) después de detectar la pulsación del botón para evitar lecturas repetidas debidas al "rebote" del botón.

Código "DEEPSLEEP + Contador"

```
## Deep Sleep + contador
import esp32
from machine import Pin, deepsleep, RTC
from time import sleep

# Configuración del RTC
rtc = RTC()

# Configuración de botones
stop_button = Pin(4, Pin.IN, Pin.PULL_UP) # Botón para detener y entrar en deep sleep
start_button = Pin(12, Pin.IN, Pin.PULL_UP) # Botón para despertar y continuar

# Configuración del pin para despertar desde deep sleep
esp32.wake_on_ext0(pin=start_button, level=esp32.WAKEUP_ALL_LOW)

# Verificar si hay un contador guardado en el RTC
if rtc.memory():
    try:
        contador = int(rtc.memory()) # Cargar el último número guardado
        print("-----")
        print("ESP-32 preparada y lista para contar")
        print(f"Nos quedamos en: {contador}") # Mensaje al despertar
    except ValueError:
        contador = 1 # Si hay un error en el RTC, reinicia el contador
else:
    contador = 1 # Si no hay nada guardado, inicia desde 1

# Comenzar o continuar el conteo
while contador <= 100:
    print("Contador:", contador)
    contador += 1 # Incrementa el contador

# Verifica si el botón de detener se presiona
if stop_button.value() == 0: # Botón presionado (nivel bajo)
    print(f"Guardando progreso en {contador} y entrando en deep sleep...")
    print("-----")
    rtc.memory(str(contador)) # Guarda el progreso en el RTC
    sleep(0.1) # Prevención de rebotes
    deepsleep() # Entra en deep sleep hasta que se despierte con el otro botón

    sleep(1) # Pausa entre números para facilitar la visualización

print("¡El contador ha terminado!")
```

Configuración de los botones:

- **stop_button (Pin 4):** Al presionarlo, el ESP32 guarda el progreso en el RTC y entra en **deep sleep**.
- **start_button (Pin 12):** Configurado con `wake_on_ext0` para que despierte al ESP32 cuando lo presiones.

Uso del RTC para guardar el progreso:

- El progreso actual ("contador") se guarda como una cadena en el RTC antes de entrar en **deep sleep**.
- Al despertar, se recupera el número guardado y el ESP32 continúa contando desde ese valor.

Mensajes claros en la consola:

- Al despertar, el dispositivo imprimirá un mensaje como "Nos quedamos en: ..." indicando el último número guardado en el RTC.

Prevención de rebotes:

- Se usa un retraso breve (`sleep(0.1)`) después de detectar un botón presionado para evitar múltiples detecciones no deseadas.

Flujo principal del programa:

- El contador sigue aumentando mientras no se presione el botón de detener.
- Si se presiona el botón de detener, el ESP32 guarda el progreso, entra en **deep sleep** y espera a ser despertado por el otro botón.

Finalización del conteo:

- El programa termina cuando el contador llega a 100, imprimiendo "¡El contador ha terminado!".

Notas Importantes:

- **Persistencia del progreso:** Gracias al RTC, el último número contado se conserva incluso después de entrar y salir del modo **deep sleep**.
- **Ajuste de pines:** Si los pines 4 y 12 no son adecuados, ya sea por el modelo de la ESP o alguna otra cosa, puedes usar otros pines como 14, 27, 33, etc.
- **Depuración:** Asegurarse de que el hardware esté bien conectado y los botones funcionen correctamente.